Отчёт по постобработке OCR паспортов

1. Задача программы

Программа получает результаты OCR для паспортных документов и решает, можно ли принять документ или безопаснее его отклонить. При решении учитываются не только распознанные значения, но и уверенность OCR, согласованность визуальной зоны документа и машиночитаемой зоны, формат полей и признаки возможных ошибок.

Основной приоритет - не принимать документ с ошибками. Поэтому сомнительный документ может получить решение reject, даже если большая часть полей выглядит правдоподобно. Это намеренно строгий подход: ошибочный приём считается более серьёзным, чем отклонение документа, который мог быть распознан верно.

2. Какие данные использует решение

В программе используются три группы входных данных:

- train_predictions.jsonl - результаты OCR для обучающих документов
- train_labels.jsonl - правильные значения полей для этих документов. Они нужны, чтобы оценить ошибки OCR и обучить классификатор
- reference/subdivisions.csv - справочник кодов подразделений и вариантов названий органов выдачи

Для обработки документов также используется JSONL-файл, указанный при запуске. Он может содержать test_predictions.jsonl или train_predictions.jsonl. Для каждого документа программа читает поля VIZ (визуально распознаваемую область) и MRZ (машиночитаемую зону), а также сведения о качестве изображения.

3. Какие поля проверяются

Для каждого документа формируются десять итоговых полей:

- Фамилия, имя и отчество (ФИО)
- Пол и дата рождения
- Место рождения
- Дата выдачи паспорта
- Код подразделения и название органа выдачи
- Серия и номер паспорта (Licence_number в структуре задания)

Текст нормализуется перед сравнением: убираются лишние пробелы и регистр букв приводится к единому виду. Это помогает не считать различия в оформлении текста ошибками.

4. Как программа принимает решение

Шаг 1. Объединяем VIZ и MRZ

Для структурированных полей программа сопоставляет распознанное значение из визуальной зоны с данными MRZ. MRZ содержит контрольные цифры: по ним можно проверить отдельные значения, например, номер документа и дату рождения. Если источники согласуются, это повышает доверие к результату. Если значения расходятся, программа учитывает конфликт и не считает его обычным совпадением.

Если поле не удалось надёжно получить из одного источника, может использоваться значение из другого, но отдельно фиксируется, насколько уверенно оно подтверждено. Для имён дополнительно сравнивается транслитерация кириллицы и латиницы. Автоматическое исправление имени само по себе не считается достаточным основанием для принятия документа.

Шаг 2. Применяем строгие проверки

Перед применением модели документ должен пройти базовые проверки. В частности, программа отклоняет документ, если:

- не удалось получить одно или несколько обязательных полей
- имя из VIZ противоречит имени из MRZ или потребовалось исправление имени
- паспорт недействителен на контрольную дату 1 октября 2026 года
- дата выдачи указывает на выдачу паспорта раньше достижения 14 лет

Для даты действия паспорта учитываются дата рождения, дата выдачи и сроки замены в 20 и 45 лет. Если обязательные даты нельзя корректно разобрать, надёжно подтвердить действительность паспорта не получается.

Шаг 3. Оцениваем риск ошибки моделью

Документы, прошедшие строгие правила, описываются набором признаков. В него входят уверенность символов OCR, число пропусков и строк в полях, качество изображения, источник снимка, признаки MRZ, согласованность полей, а также особенности текста места рождения и органа выдачи.

На этих признаках обучается модель “HistGradientBoostingClassifier” из библиотеки scikit-learn. Для оценки её работы используется кросс-валидация с пятью частями, которая повторяется два раза. При каждой проверке модель обучается на одних частях данных, а оценивается на оставшейся части. На основе полученных результатов выбирается порог, при котором документ принимается.

При выборе порога действует штраф 50 за ошибочно принятый документ. Это делает критерий принятия строгим: программа предпочитает сократить число принятых документов, если это помогает уменьшить риск ошибки.

5. Результат работы и ограничения

Результат сохраняется в answers.jsonl - по одной JSON-записи на документ. Если документ принят, запись содержит doc_id, решение accept и все десять полей. Если документ отклонён, запись содержит doc_id и решение reject без распознанных полей.

Во время работы программа также выводит:

- сколько обучающих документов прошло строгие проверки и сколько среди них имело ошибки
- выбранный порог и результат перекрёстной проверки
- число обработанных документов, прошедших строгие правила, и число принятых
- ориентировочный прогноз ошибок среди принятых документов и грубую верхнюю оценку.

Прогноз ошибок рассчитывается по результатам проверки на обучающих данных. Это не фактическое число ошибок в test: у тестовой выборки нет доступной разметки для такого подсчёта. Чтобы измерить реальное качество, нужны правильные ответы для отдельной выборки, которая не использовалась при обучении и выборе порога.

6. Структура проекта

```
project/
├── postprocess.py
├── evaluate.py
├── requirements.txt
├── data/
│   ├── train_predictions.jsonl
│   ├── train_labels.jsonl
│   └── test_predictions.jsonl
└── reference/
    └── subdivisions.csv
```

Файл requirements.txt фиксирует версии основных библиотек:

1. numpy 2.2.6
2. pandas 2.2.3
3. scikit-learn 1.6.1

Пути к обучающим данным и справочнику программа находит относительно расположения postprocess.py, поэтому запускать её следует из корневой папки проекта.

7. Как запустить программу

Создайте виртуальное окружение и активируйте его в Windows:

- python -m venv .venv
- .venv\Scripts\activate

Установите зависимости:

- python -m pip install -r requirements.txt

Для обработки тестовых предсказаний выполните:

- python postprocess.py data/test_predictions.jsonl answers.jsonl

Для обработки обучающих предсказаний используйте:

- python postprocess.py data/train_predictions.jsonl answers.jsonl

Другие параметры командной строки не нужны. При обоих вариантах программа автоматически загружает train_predictions.jsonl, train_labels.jsonl и reference/subdivisions.csv для обучения и проверки, а результат записывает в answers.jsonl.

Важно: запуск с train_predictions.jsonl в качестве входного файла предназначен для обработки этого файла, но не является независимой проверкой качества - эти документы также участвуют в обучении модели.